

RobustGaSP: Bayesian vs MLE

Gu (2019)

Examples

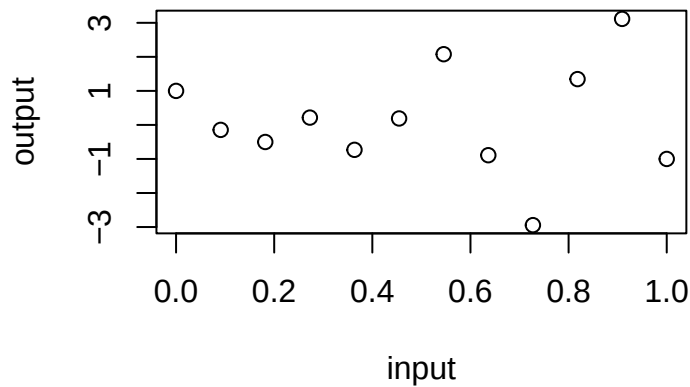
The modified sine wave function

It is expected that, for a one-dimensional function, both packages will perform well with an adequate number of design points, so we start with the function called the modified sine wave discussed in Gu (2016). It has the form

$$y = 3\sin(5\pi x) + \cos(7\pi x),$$

where $x = [0, 10]$. We first perform emulation based on 12 equally spaced design points on $[0, 1]$.

```
sinewave <- function(x) {  
  3*sin(5*pi*x)*x + cos(7*pi*x)  
}  
input <- as.matrix(seq(0, 1, 1/11))  
output <- sinewave(input)  
plot(output ~ input)
```



The GaSP model is attempted to be fitted with the RobustGaSP package, using the marginal posterior mode for estimation (the default in RobustGaSP) first, then maximum likelihood estimation second, both time with a Gauss kernel.

```
m.pm <- rgasp(design=input, response=output,  
  kernel_type = 'pow_exp',  
  alpha = rep(2, ncol(input)),  
)
```

```
## The upper bounds of the range parameters are 0.7378104
## The initial values of range parameters are 0.01475621
## Start of the optimization 1 :
## The number of iterations is 8
## The value of the marginal posterior function is -19.61828
## Optimized range parameters are 0.04123854
## Optimized nugget parameter is 0
## Convergence: TRUE
## The initial values of range parameters are 0.01388889
## Start of the optimization 2 :
## The number of iterations is 8
## The value of the marginal posterior function is -19.61828
## Optimized range parameters are 0.04123854
## Optimized nugget parameter is 0
## Convergence: TRUE
```

```
m.pm
```

```
##
## Call:
## rgasp(design = input, response = output, kernel_type = "pow_exp",
##       alpha = rep(2, ncol(input)))
## Mean parameters: 0.1440581
## Variance parameter: 2.543004
## Range parameters: 0.04123854
## Noise parameter: 0
```

```
m.mle <- rgasp(design=input, response=output,
               kernel_type = 'pow_exp',
               alpha = rep(2, ncol(input)),
               method = "mle"
               )
```

```
## The upper bounds of the range parameters are 0.7378104
## The initial values of range parameters are 0.01475621
## Start of the optimization 1 :
## The number of iterations is 1
## The value of the profile likelihood function is -19.97904
## Optimized range parameters are 0.01475621
## Optimized nugget parameter is 0
## Convergence: TRUE
## The initial values of range parameters are 0.01388889
## Start of the optimization 2 :
## The number of iterations is 1
## The value of the profile likelihood function is -19.97904
## Optimized range parameters are 0.01388889
## Optimized nugget parameter is 0
## Convergence: TRUE
```

```
m.mle
```

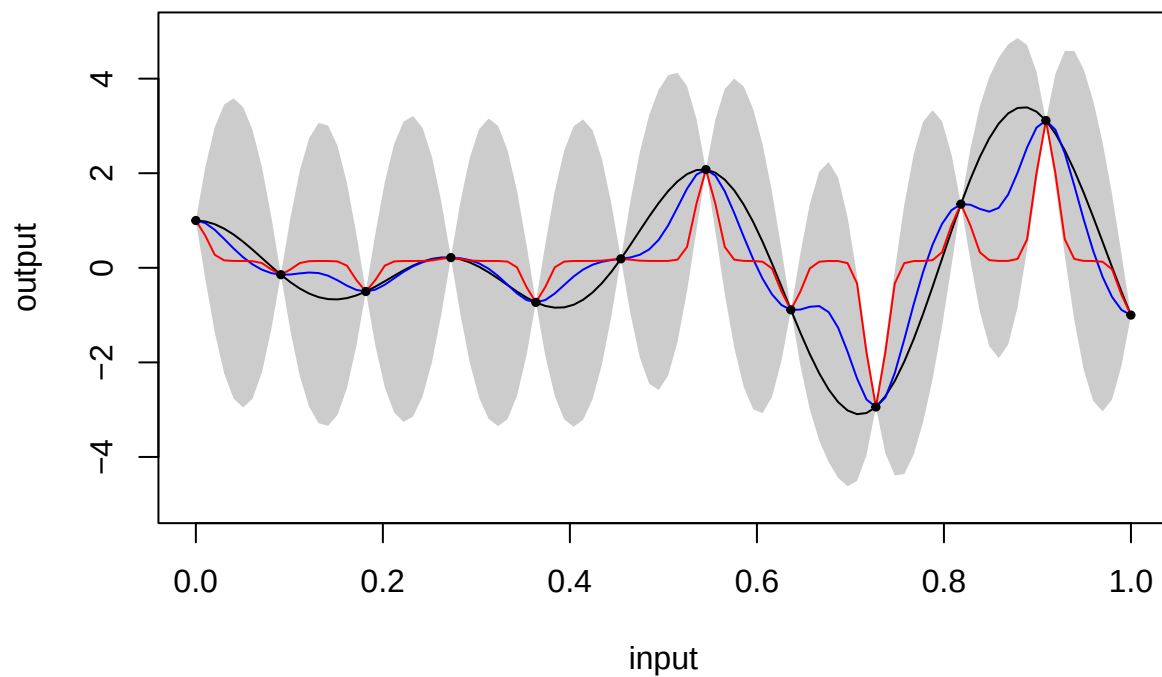
```
##
```

```
## Call:
## rgasp(design = input, response = output, method = "mle", kernel_type = "pow_exp",
##       alpha = rep(2, ncol(input)))
## Mean parameters: 0.1442577
## Variance parameter: 2.327824
## Range parameters: 0.01475621
## Noise parameter: 0
```

Using marginal posterior mode, the estimated range parameter is found to be around 0.04 in the **RobustGaSP** package and 0.01 when using MLE. To see which estimate is better, we perform prediction on 100 test points, equally spaced in $[0,1]$.

```
testing_input <- as.matrix(seq(0, 1, 1/99))
m.pm.predict <- RobustGaSP::predict(
  m.pm,
  testing_input
)
m.mle.predict <- RobustGaSP::predict(
  m.mle,
  testing_input
)
testing_output <- sinewave(testing_input)
```

The emulation results are plotted in Figure 2.



Note that the red curve from the **DiceKriging** package degenerates to the fitted mean with spikes at the design points. This unsatisfying phenomenon, discussed in Gu et al. (2018), happens when the estimated

covariance matrix is close to an identity matrix, i.e., $\hat{\mathbf{R}} \approx \mathbf{I}_n$, or equivalently $\hat{\gamma}$ tends to 0. Repeated runs of the `DiceKriging` package under different initializations yielded essentially the same results.

Table 1: Estimates table with 12 observations

RobustGaSP method	Kernel	Trend	Range	Variance
Marginal posterior mode	Gauss	0.14	0.04	2.54
Maximum likelihood estimation	Gauss	0.14	0.01	2.33

To compare the performance, we calculate the root mean square errors (RMSE) for both methods,

$$RMSE = \sqrt{\frac{\sum_{i=1}^{n^*} (\hat{y}(\mathbf{x}_i^*) - y(\mathbf{x}_i^*))^2}{n^*}},$$

where $y(\mathbf{x}_i^*)$ is the i -th held-out output and $\hat{y}(\mathbf{x}_i^*)$ is the prediction for $\mathbf{x}_i^*$ by the emulator, for $i = 1, \dots, n^*$.

```
m.pm.rmse <- sqrt(mean((m.pm.predict$mean - testing_output)^2))
m.pm.rmse
```

```
## [1] 0.6053422
```

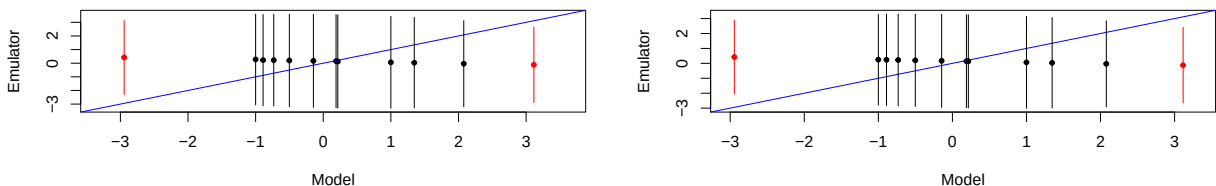
```
m.mle.rmse <- sqrt(mean((m.mle.predict$mean - testing_output)^2))
m.mle.rmse
```

```
## [1] 1.210554
```

```
plot(m.pm.predict$mean ~ testing_output,
     col="blue", pch=19, cex=0.25,
     xlab = "model", ylab = "emulator")
points(m.mle.predict$mean ~ testing_output,
       col="red", pch=19, cex=0.25)
abline(0,1)
```

The RMSE when using the marginal posterior mode for estimation 0.6053422 and when using maximum likelihood estimation is 1.2105539. The predictions versus the real outputs are plotted in Figure 8. The blue dots correspond to the predictive means using the marginal posterior mode and are closer to the real output than the red dots produced when using maximum likelihood estimation. Since both packages use the same correlation and mean function, the only difference lies in the method of parameter estimation, especially estimation of the range parameters, γ . Use of the marginal posterior mode seems to do better, leading to much smaller RMSE in out-of-sample prediction.

Next we'll perform LOO cross validation.



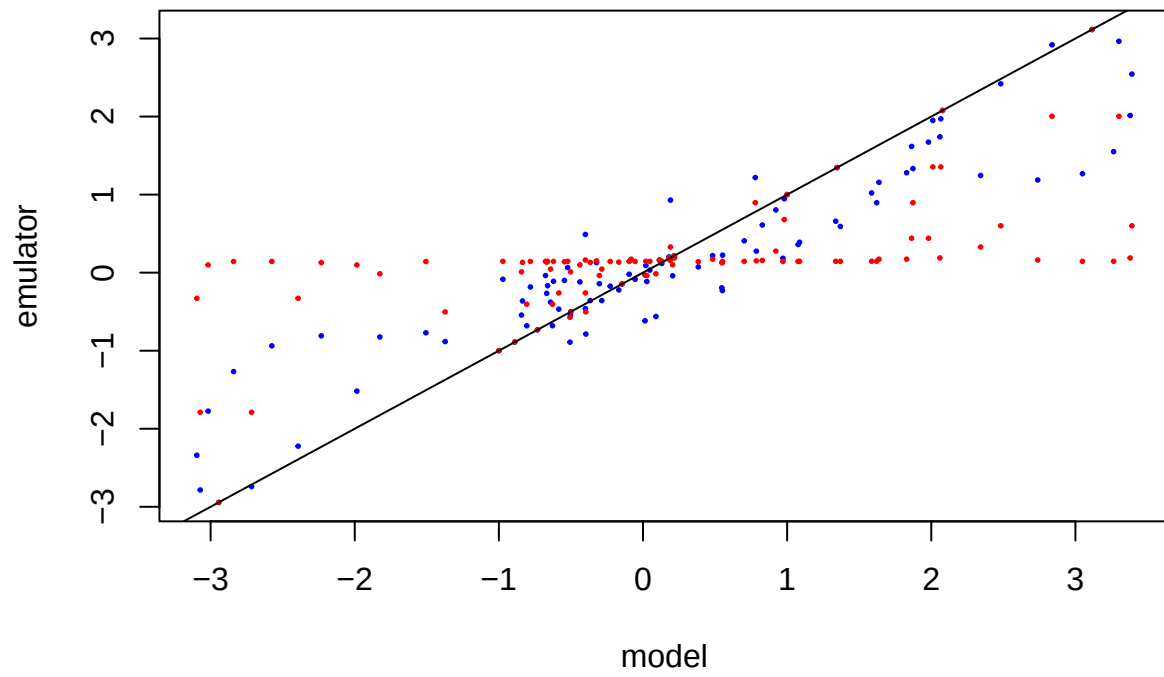


Figure 1: Figure 8

Verifying the estimates by-hand for both packages

To calculate the posterior predictions at the 100 testing points by-hand, we require the matrix H containing the inputs for the observations and well as matrix A , its inverse, and $\mathbf{t}(x^*)$ for all 100 of the new inputs x^* . A and $\mathbf{t}(x^*)$ are built using the estimate of the range parameter returned by each package. By-hand predictions match those returned by `DiceKriging` (Figure 2) and `RobustGaSP` (Figure 3).

```
## [1] 0.1440581
```

```
## [1] 0.04123854
```

```
## [1] 2.543004
```

```
## [1] 0.1442577
```

```
## [1] 0.01475621
```

```
## [1] 2.327824
```